AIF-C01 Study Guide

Oct 29, 2026

Exam facts

You need a scaled score of 700 out of 1,000 to pass AIF-C01; most of the exam rewards knowing the fundamentals and matching scenario keywords to the right AWS service.

- **Format:** 65 questions (50 scored, 15 unscored), 90 minutes, multiple choice, multiple response, ordering and matching. No penalty for guessing, so never leave one blank.
- **Retakes:** after a failed attempt you must wait 14 days. Each attempt is a new fee unless you have a voucher or promotion.
- **Score report:** shows pass or fail, your scaled score, and each domain marked "Needs improvement" or "Meets competencies."
- **Practice tests:** randomized tests tend to predict the real exam better than timed sets you have seen before. Aim for 75% or higher on fresh questions.

Domain	Weight
1. Fundamentals of AI and ML	20%
2. Fundamentals of generative AI	24%
3. Applications of foundation models	28%
4. Guidelines for responsible AI	14%
5. Security, compliance and governance	14%

How to use this guide: sections 2 to 6 follow the five exam domains; sections 7 and 8 are one-page lookups for metrics and services; section 9 lists tricky practice questions and the rule that answers each.

Domain 1: Fundamentals of AI and ML (20%)

Most Domain 1 questions ask you to name the learning type, the problem type, the lifecycle stage or the SageMaker tool from a short scenario.

AI, ML, deep learning, generative AI

Each is a subset of the one before it.

- **AI:** any system doing tasks that normally need human intelligence.
- **Machine learning:** learns patterns from data instead of hand-written rules.
- **Deep learning:** ML with many-layered neural networks; strongest on images, audio and text.
- **Generative AI:** deep learning that creates new content. Output is new content → generative AI; output is a label or a number → traditional ML.

Learning types

Type	Data	Example	Tell in the question
Supervised	Labeled	Loan default yes/no, house price	"historical data labeled…"
Unsupervised	Unlabeled	Customer segments (clustering), anomaly detection	"no predefined labels", "group"
Reinforcement	Rewards and penalties	Game AI, robotics, AWS DeepRacer	"agent", "reward", "trial and error"
Self-supervised	Unlabeled; the data makes its own labels	Predict the next word: "The cat sat on the ___" → "mat"	how foundation models are pre-trained
RLHF	Human rankings become the reward	Aligning an LLM to be helpful and safe	"humans rank responses"

Unsupervised techniques (no labels; find hidden structure):

Technique	Finds	Tell in the question
Clustering (k-means)	Groups of similar items	"group users with similar habits", "segment"
Association rule learning	Items that occur together	"shows watched together", "bought together", market basket

Technique	Finds	Tell in the question
Probability density estimation	How data is distributed	"how preferences are spread across genres"
Anomaly detection	Rare, unusual points	"unusual transactions", outliers
Dimensionality reduction (PCA)	Fewer features keeping most information	"reduce the number of features"

Decision trees, linear and logistic regression, and (as Tutorials Dojo frames it) neural networks are supervised: wrong answers for unsupervised tasks.

Reinforcement learning's risk-versus-reward trade-off is called **exploration vs. exploitation**: try something new or stick with what works.

Problem types

- **Classification:** predicts a category (spam / not spam, fraud / not fraud).
- **Regression:** predicts a number (price, demand, temperature).
- **Clustering:** finds natural groups with no labels.

Model fit

Symptom	Diagnosis	Fix
Great on training data, poor on test data	Overfitting (high variance)	More data, regularization, early stopping, simpler model
Poor on both	Underfitting (high bias)	More complex model, better features, train longer

- **Hyperparameters** are set before training: learning rate, epochs, batch size, tree depth. SageMaker Automatic Model Tuning searches them.
- **Parameters** (weights) are learned during training.
- **Training / validation / test split:** train on one part, tune on validation, report the final score on test data the model never saw.

Common algorithms

Algorithm	Type	Pick when
Decision tree	Supervised (classification or regression)	Interpretable, hierarchical breakdown of how each factor drives the result
Linear regression	Supervised regression	Predict a number; simple and interpretable
Logistic regression	Supervised classification	Yes/no probability; linear, interpretable coefficients
k-NN	Supervised classification	Classify by the nearest labeled examples (seashell type from its attributes)
k-means	Unsupervised clustering	Group unlabeled data into segments
XGBoost, random forest	Supervised ensembles	High accuracy on tabular data
SVM	Supervised classification	Max-margin boundary; hard to interpret
Factorization machines	Supervised, sparse data	Recommendations, click prediction
CNN	Deep learning	Images
RNN, LSTM	Deep learning	Sequences, time series
Transformer	Deep learning	Language, LLMs

Epochs: one epoch is one full pass over the training data. A fine-tuned model under its accuracy target → increase epochCount (too many risks overfitting).

k-NN vs. k-means (both use distance and a "k"): k-NN is supervised; it finds the k most similar labeled examples and they vote on the label ("predict the seashell type"). k-means is unsupervised; it groups unlabeled points around k centers and the groups have no names ("segment customers"). Hook: k-**N**N = **N**eighbors vote on a **N**ame; k-**M**eans = the **M**iddle of groups.

ML lifecycle (AWS Well-Architected ML Lens)

1. **Business goal:** the problem and its business value or ROI.
2. **Frame the ML problem:** target variable, problem type, and which metric and error level count as success.
3. **Data processing:** collect, clean, label, engineer features (for example, deriving "day of week" from a timestamp).
4. **Model development:** train, tune, evaluate against the metric from step 2.
5. **Deploy.**
6. **Monitor** for drift, then retrain (back to step 3).

The success metric is decided in step 2 and measured in step 4. Data quality problems are fixed at data collection.

Inference options (SageMaker)

Option	Pick when
Real-time endpoint	Low latency, steady traffic, one request at a time
Serverless	Intermittent traffic with idle periods; scales to zero; cold starts possible
Asynchronous	Large payloads or long processing (minutes, up to about an hour); clients can wait
Batch transform	Whole dataset on a schedule; latency does not matter

SageMaker tools

Tool	Does
Ground Truth	Human labeling of training data
Data Wrangler	Data preparation and transformation
Feature Store	Stores features for reuse in training and inference
Canvas	No-code ML for business analysts

Tool	Does
JumpStart	Pre-trained models and solutions you deploy and host
Automatic Model Tuning	Hyperparameter search
Clarify	Bias detection and explainability (SHAP)
Model Monitor	Drift and quality checks in production
Model Registry	Versions and approves models
Pipelines	Automates the ML workflow (MLOps)
Model Cards	Documents a model for governance
Autopilot	AutoML: trains and tunes models automatically (distractor for documentation or explainability)
Debugger	Catches training problems (vanishing gradients, overfitting) in real time
Smart sifting	Filters low-value training samples to speed training
Ground Truth Plus	Fully managed labeling with an AWS-provided workforce
HyperPod	Resilient clusters for long training runs of very large foundation models
Neo	Compiles and optimizes models to run faster on edge devices

Managed AI services

Service	Does
Comprehend	Text: sentiment, entities, key phrases, language, PII
Textract	Extracts text, handwriting, tables, forms from documents
Rekognition	Images and video: faces, objects, content moderation

Service	Does
Transcribe	Speech → text
Polly	Text → speech
Translate	Language translation
Lex	Chatbots and voice bots
Kendra	Intelligent enterprise search
Personalize	Recommendations
Forecast	Time-series forecasting
Fraud Detector	Online fraud detection
Comprehend Medical	Medical text: extracts conditions, medications, dosages, procedures, their relationships, and PHI

Pipelines show up too: call recordings → Transcribe, then Comprehend for sentiment. Photos of documents, notes or labels read aloud → Textract, then Polly. Structured patient data summarized → Comprehend Medical plus rule-based logic (not Textract: nothing scanned; not QuickSight: dashboards only). Rekognition can spot text in a photo, but documents and handwriting → Textract.

Domain 2: Fundamentals of generative AI (24%)

Domain 2 tests vocabulary and trade-offs: what tokens, embeddings and context windows are, which model type fits, and where generative AI helps or fails.

Core concepts

Term	Meaning
Token	A piece of text (word or word part). Drives cost, length limits and the context window
Embedding	A numeric vector capturing meaning; similar meanings sit close together. Powers semantic search and RAG
Vector	The list of numbers an embedding is stored as

Term	Meaning
Context window	Most tokens (input + output) a model can consider at once; anything beyond is cut off
Transformer	The architecture behind LLMs; uses self-attention to weigh how tokens relate
Prompt / completion	Your input / the model's output
Hallucination	Confident, plausible, false output
Nondeterminism	The same prompt can give different answers
Foundation model	Huge model pre-trained (self-supervised) on broad unlabeled data, adaptable to many tasks
Tokenization	Splitting text into tokens before processing

Tokens are pieces of text; embeddings are numbers that represent meaning.

Model types

Model	Does	Rule
LLM	Generates or understands text	"generate text or documentation"
Diffusion (Stable Diffusion, Titan Image Generator)	Generates images from noise, refined step by step	"create images"
Multimodal	Several data types together: text, images, audio	"read a photo and answer in text"
Foundation model	General base, fine-tuned for many task types	"adapt to many different tasks"
Embedding model (Titan Embeddings)	Turns text into vectors	"semantic search", "similar documents"
GAN	Generator vs. discriminator compete	older image generation

AWS's own model families appear in question stems: Amazon Nova (Micro, Lite, Pro for text and multimodal; Canvas for images; Reel for video) and Amazon Titan (Text Express and Lite, Embeddings, Image Generator). Third-party models on Bedrock include Anthropic Claude, Meta Llama, Mistral, Cohere and Stability AI.

Foundation model characteristics:

Characteristic	Means	Question says…
Scalability	Performance grows with size (parameters, data); needs massive compute	"model size", "parameters", "emergent behaviors"
Adaptability and generalization	One model, many tasks, no task-specific training	"many different tasks", "zero-shot, few-shot"
Modality	The types of data it takes in and produces	"text, images, or both", "multimodal"
Customization and architecture	How it is built and specialized	"transformer, diffusion", "fine-tuning, RAG, RLHF"

Capabilities and challenges

Capabilities (advantages)	Challenges specific to generative AI	Challenges for all AI
Personalization	Hallucination	Privacy
Adaptability	Toxicity	Explainability
Scalability	Intellectual property, copyright	Bias
Simplicity (natural-language interface)	Nondeterminism	Cost
Speed of content creation	Plagiarism, misuse	

Common generative AI disadvantages in production: hallucination, knowledge cutoff, bias in outputs, lack of explainability (black box), overreliance on AI, security and privacy concerns, resource and cost intensity.

Capabilities are what the system does; metrics are what you measure. Customer lifetime value and cross-domain performance are metrics, never capabilities.

When not to use generative AI: tasks needing exact, deterministic, auditable answers (payroll math, regulatory calculations). Use traditional software.

Choosing and paying for a model

- **Selection factors:** quality, latency, cost, context window size, modality, language support, open vs. proprietary, customization options.
- **Smaller model:** faster and cheaper; right when speed and volume matter more than top quality.
- **Larger model:** more capable, slower, more expensive.
- **On-demand pricing:** per input and output token; good for variable or low traffic. Cut cost with shorter prompts, lower max tokens, a smaller model.
- **Provisioned Throughput:** reserved capacity for steady high traffic; required for fine-tuned or custom models.
- **Batch inference:** cheaper for large offline jobs.

Which service

Need	Service
Managed API to foundation models from many providers, nothing to host	Amazon Bedrock
Pick a pre-trained model and host it yourself	SageMaker JumpStart
Ready-made assistant over company documents and apps	Amazon Q Business
Coding and AWS help in the IDE and console	Amazon Q Developer
No-code playground to try generative AI apps, no AWS account	PartyRock
Build gen AI apps with no code inside SageMaker	SageMaker Canvas

Business metrics for generative AI

Conversion rate, average handle time per ticket, customer satisfaction, cost per user or interaction, revenue per user, productivity gains, efficiency. Model metrics (ROUGE, perplexity) are different; see section 7.

Domain 3: Applications of foundation models (28%)

Domain 3 is the largest at 28%; it centers on the customization ladder, prompting and Bedrock features.

Customization ladder (cheapest first)

Approach	Changes weights?	Data	Pick when
Prompt engineering	No	None	Always try first
RAG	No	Your documents in a vector database	Need current or private facts, fewer hallucinations, citations
Fine-tuning	Yes	Small, labeled (prompt–response pairs)	Change style, tone, format or task behavior
Continued pre-training	Yes	Unlabeled domain data	Model lacks your domain's vocabulary (medical, legal)
Train from scratch	Builds new	Massive, unlabeled	Almost never the answer

"Labeled examples" → fine-tuning. "Unlabeled domain data" → continued pre-training. A fine-tuned Bedrock model needs Provisioned Throughput.

RAG in five steps

1. Split documents into chunks.
2. Turn each chunk into an embedding (Titan Embeddings).
3. Store the vectors in a vector database.
4. At question time, retrieve the most similar chunks.

5. Add them to the prompt; the model answers from them.

On AWS this is **Bedrock Knowledge Bases**. Vector stores: OpenSearch Serverless (Bedrock default), Aurora or RDS PostgreSQL with pgvector, Neptune Analytics, MemoryDB, DocumentDB, plus third-party (Pinecone, Redis, MongoDB Atlas). Wrong answers: S3, Redshift, QuickSight, DynamoDB. Tutorials Dojo lists "Neptune ML"; the real vector feature is Neptune Analytics.

Keeping a Knowledge Base current: upload new documents to its S3 data source, then sync (resync) the Knowledge Base. No retraining. Questions that evolve over time at low cost → RAG plus prompt engineering, never continuous retraining.

Prompting techniques

Technique	What it is	Tell
Zero-shot	No examples; relies on general knowledge	"no examples provided"
One-shot	One example	"a single example"
Few-shot	Several examples in the prompt	"provide a few labeled examples"
Chain-of-thought	Reason step by step	"break into steps", "show reasoning"
Self-refine	Model critiques and revises its own answer	"review and improve its output"
Prompt template	Reusable structure with placeholders	"standardize prompts"
Negative prompting	Say what to avoid	"do not include…"

More techniques that appear as answers or distractors: **least-to-most** (solve smaller subproblems in order), **tree-of-thought** (explore branching reasoning paths), **prompt chaining** (output of one prompt feeds the next), **adversarial prompting** (test with malicious inputs to harden against injection), **negative prompts** (say what an image must not contain), and **CFG scale** for image models (higher = closer to the prompt, more detail).

Good prompts state the role, task, context, output format and constraints.

Prompt risks

- **Prompt injection:** malicious instructions hidden in input.
- **Jailbreaking:** tricking the model past its safety rules.
- **Prompt leaking:** making the model reveal its system prompt.
- **Poisoning:** corrupting training data.
- **Hijacking:** steering the model to an unintended task.

Defend with Bedrock Guardrails, input validation and least-privilege access.

Inference parameters

Parameter	Effect
Temperature	Higher = more random and creative; lower = focused and consistent
Top-p	Samples from the smallest set of words whose probabilities add up to p
Top-k	Samples only from the k most likely words
Max tokens	Caps output length (and cost)
Stop sequences	End output when a set string appears

To reduce hallucination or variation: lower temperature, plus RAG for grounding.

Bedrock features

Feature	Does
Knowledge Bases	Managed RAG
Agents	Multi-step tasks; calls APIs and Lambda functions through action groups
Guardrails	Content filters, denied topics, PII redaction, word filters, contextual grounding checks
Model Evaluation	Compare models: automatic metrics or human reviewers

Feature	Does
Custom models	Fine-tuning and continued pre-training
Provisioned Throughput	Reserved capacity; required for custom models
Prompt management, Flows	Store prompts; chain steps into workflows

Guardrails policy types: content filters (hate, insults, sexual, violence, misconduct, prompt attacks), denied topics, word filters, sensitive information filters (PII), contextual grounding checks. **Model invocation logging** saves full prompts and responses to CloudWatch Logs or S3.

Content filter categories mnemonic: **H**is **I**nsults **S**tarted **V**iolent **M**isconduct **P**roblems (hate, insults, sexual, violence, misconduct, prompt attack). Gambling → denied topic; misinformation → contextual grounding; "discrimination" is not a category name.

Customizing a model in Bedrock, in order:

1. Prepare the training dataset (labeled for fine-tuning, unlabeled for continued pre-training) in S3.
2. Create the fine-tuning or continued pre-training job.
3. Purchase Provisioned Throughput to run the custom model.

Evaluating results, setting max tokens and adding KMS keys are useful but not required steps. Regularly updating an FM with new unlabeled data → continued pre-training.

Look-alikes: fine-tuning = adapt a pre-trained model with labeled data (pick it over the broader "transfer learning"); hyperparameter tuning = adjust training settings; model quantization = shrink a model for speed, teaches it nothing. Primary benefit of fine-tuning = better performance on your use case. **Pricing:** occasional or unpredictable traffic on a base model → On-Demand; custom or fine-tuned model, or steady traffic → Provisioned Throughput.

Foundation model capabilities

When matching, pick the broad capability (visual comprehension, language processing, speech to text, code generation), not a narrow application (image classification, translation).

Evaluating a foundation model

Human evaluation (judgment, tone, helpfulness), benchmark datasets, automatic metrics (section 7), and business metrics. Bedrock Model Evaluation supports both automatic and human-review jobs.

Domain 4: Guidelines for responsible AI (14%)

Domain 4 comes down to naming the responsible-AI dimension at stake and the AWS tool that addresses it.

Dimensions

Dimension	Means
Fairness	Equitable outcomes across groups
Explainability	Can say why the model made a decision
Interpretability	Humans can understand how the model works inside
Transparency	Open about how the system works, its data and limits
Robustness	Reliable under noisy, changed or adversarial inputs
Privacy and security	Protects personal data
Safety	Avoids harmful output
Controllability	Humans can monitor and steer behavior
Governance	Policies, accountability, documentation
Veracity	Outputs are accurate and truthful

Keyword → dimension:

If the question says…	Answer
"clarity on how decisions are made", "visible to stakeholders"	Transparency

If the question says…	Answer
"why this specific prediction", "which features drove it"	Explainability
"unbiased", "doesn't discriminate", "equal treatment"	Fairness
"reliable", "consistent", "withstands adversarial inputs"	Veracity and robustness
"protect personal data", "PII"	Privacy and security
"policies", "frameworks", "accountability"	Governance
"avoid harm", "unintended consequences"	Safety
"human oversight", "can override the AI"	Controllability

Transparency vs. explainability: transparency is about the whole system (open about how it works, its data and limits; Model Cards). Explainability is about one decision (why this person got this result; Clarify, SHAP). A welfare-eligibility question asking for "clarity on how decisions are made" is transparency; fairness is the trap.

Bias types

Bias	Cause
Sampling bias	Training data over- or under-represents groups
Automation bias	People trust AI output too much
Measurement bias	Data collected inaccurately or inconsistently
Confirmation bias	Favoring data that supports what you expect
Historical bias	Data reflects past unfair decisions

Fix with diverse, balanced, representative data, measured with SageMaker Clarify before and after training.

Tools

Tool	Use
SageMaker Clarify	Bias detection and explainability (SHAP feature importance)
Bedrock Guardrails	Filter harmful content, deny topics, redact PII, grounding checks
Amazon A2I	Human review of low-confidence predictions
SageMaker Ground Truth	Human labeling of training data (incl. RLHF data)
SageMaker Model Cards	Document your model: intended use, data, evaluation, risk
AWS AI Service Cards	AWS's own documentation of its AI services
SageMaker Model Monitor	Watch production for drift and bias drift
Bedrock Model Evaluation	Measure toxicity, accuracy, robustness before launch

Interpretability trade-off

Simpler models (linear regression, decision trees) are easy to explain; complex models (deep neural networks) can be more accurate but act as black boxes. When a regulator requires explaining each decision, choose the interpretable model.

Generative AI legal and ethical risks

Intellectual property and copyright infringement, biased output, toxicity, hallucination, loss of customer trust, and end-user risk. Mitigate with content filters, human review, licensed training data and clear disclosure that content is AI-generated.

Good datasets

Diverse, inclusive, balanced across groups, curated, accurate, and representative of the people the model will serve.

Domain 5: Security, compliance and governance (14%)

Standard AWS security (IAM, KMS, CloudTrail) carries over; the exam adds AI-specific rules and several look-alike services.

Shared responsibility for AI

- **AWS (security of the cloud):** data centers, hardware, the managed Bedrock and SageMaker services, base model hosting.
- **You (security in the cloud):** IAM permissions, your data and prompts, encryption choices, network setup, how you use outputs.

Bedrock data privacy

- Bedrock does **not** use your prompts or data to train base models, and does not share them with model providers.
- Fine-tuned models are private copies in your account.
- Data stays in the Region you choose.

Controls

Need	Use
Least-privilege access to models and data	IAM roles and policies
Encrypt data with keys you control and rotate	AWS KMS customer managed keys
Keep traffic off the public internet	AWS PrivateLink (interface VPC endpoints)
S3 or DynamoDB from a VPC, free	Gateway endpoint (interface endpoint for on-premises, other Regions or other services)
Protect training data integrity	Role-based access + cryptographic hashing (versioning only records changes)
Store credentials	Secrets Manager

Look-alike services

Service	Does	Not to be confused with
AWS Artifact	Download AWS's own compliance reports (SOC, ISO, PCI) and agreements	Audit Manager
AWS Audit Manager	Collects audit evidence about your environment	Artifact
AWS Config	Continuously checks resource configuration against rules	CloudTrail
AWS CloudTrail	Logs API calls: who, what, when	CloudWatch
Amazon CloudWatch	Metrics, logs, alarms	CloudTrail
Amazon Macie	Finds PII and sensitive data in S3	Inspector
Amazon Inspector	Scans EC2, ECR, Lambda for vulnerabilities	Macie
Amazon GuardDuty	Detects threats and suspicious activity	Inspector
AWS Trusted Advisor	Best-practice checks: cost, security, fault tolerance	Config
Security Hub	Central view of security findings	GuardDuty
AWS Data Exchange	Subscribe to third-party datasets, get revision notices	Artifact

Data governance terms

Term	Means
Data lineage	Where data came from and how it was transformed
Data residency	Where data is physically stored (Region)
Data retention	How long data is kept
Data cataloging	Inventory of datasets (AWS Glue Data Catalog)

Term	Means
Data quality	Accuracy, completeness, consistency
Logging and monitoring	Recording access and use for audits

Governance practices

Policies and review boards, model documentation (Model Cards), audit trails, regular reviews, data classification, and compliance standards such as ISO 42001 (AI management) and the EU AI Act.

Metrics master sheet

First decide the task (classification, regression, text generation, or business outcome); the metric family follows from it. ROUGE is a text-generation metric, never a classification one.

Confusion matrix (classification, built after training on test data)

	Predicted positive	Predicted negative
Actually positive	True positive (TP)	False negative (FN): a miss
Actually negative	False positive (FP): a false alarm	True negative (TN)

Visualizing a multiclass model's performance → confusion matrix.

Classification

Metric	Formula	Use when
Accuracy	(TP + TN) / all	Balanced classes only; misleading when one class is rare
Precision	TP / (TP + FP)	False alarms are costly (good email flagged as spam)
Recall (sensitivity)	TP / (TP + FN)	Misses are costly (fraud, disease, security threats)

Metric	Formula	Use when
F1	Balance of precision and recall	Imbalanced classes, both errors matter
F2	Weights recall above precision	Missing a positive is worse than a false alarm
AUC-ROC	Area under the ROC curve, 0.5 to 1	Separating classes across all thresholds

Quick picker: "proportion correct across all classes" (balanced) → accuracy · rare class, imbalanced, "distinguish classes" → AUC or F1 · "avoid false alarms" → precision · "catch every positive" → recall · benchmark a fine-tuned LLM's accuracy → F1 · R^2, MAE, RMSE → regression only, always wrong for classification.

Regression

Metric	Means
MAE	Average absolute error, in the target's units
MSE	Average squared error; punishes big misses
RMSE	Square root of MSE, back in the target's units
R^2	Share of variance explained, up to 1

Text generation

Metric	Measures	Use for	Memory hook
ROUGE	Word overlap with a reference, recall-oriented	Summarization	ROUGE → Recall → summaRy
BLEU	Word overlap, precision-oriented	Translation	BLEU → bilingual
BERTScore	Meaning similarity via embeddings	Paraphrases; wording differs, meaning matches	BERT understands meaning

Metric	Measures	Use for	Memory hook
Perplexity	How surprised the model is by text; lower is better	Language model quality	low = confident

Operational and business

- **Responsiveness, runtime efficiency:** average response time. Inference latency grows with input and output length; Bedrock cost grows with tokens processed.
- **Business outcomes:** conversion rate, average handle time, customer satisfaction, cost per user or interaction, revenue per user, customer lifetime value, productivity. These measure outcomes, not model quality.

AWS service picker

Find the scenario's keyword, then the service; most service questions are lost to misreading the keyword, not to missing knowledge.

Scenario keyword	Service
Paper forms, handwriting, tables, scanned documents	Textract
Sentiment, entities, key phrases in text	Comprehend
Toxic or harmful text in comments	Comprehend toxicity detection
Medical entities from patient records	Comprehend Medical
Patient conversation → transcript + clinical summary	AWS HealthScribe
Faces, objects, unsafe images or video	Rekognition (Content Moderation)
Speech to text, call recordings	Transcribe
Text to speech	Polly
Photo of text read aloud	Textract, then Polly
Translate languages	Translate
Chatbot, voice bot	Lex

Scenario keyword	Service
Search across company documents	Kendra
Product recommendations	Personalize
Forecast future values	Forecast
Fraudulent transactions	Fraud Detector
Managed API to foundation models	Bedrock
RAG over your documents	Bedrock Knowledge Bases
Add new data to a Knowledge Base	Upload to its S3 bucket, then resync
Multi-step tasks calling APIs	Bedrock Agents
Block topics, filter harmful output, redact PII in gen AI	Bedrock Guardrails
Full prompt and response logs	Bedrock model invocation logging
Compare foundation models, measure toxicity	Bedrock Model Evaluation
Enterprise assistant over company data	Amazon Q Business
Coding assistant	Amazon Q Developer
No-code gen AI playground	PartyRock
No-code ML for analysts	SageMaker Canvas
Pre-trained models you deploy	SageMaker JumpStart
Human labeling of training data	SageMaker Ground Truth
Human review of predictions	Amazon A2I
Bias, fairness, explainability, partial dependence plots	SageMaker Clarify
Drift in production	SageMaker Model Monitor
Prepare data	SageMaker Data Wrangler
Reusable features	SageMaker Feature Store
Document a model	SageMaker Model Cards

Scenario keyword	Service
Document training and explain predictions for audits	Model Cards + Clarify
Clean and structure raw data (ETL)	AWS Glue
AWS's own compliance reports	AWS Artifact
Audit evidence about your environment	AWS Audit Manager
Continuously monitor compliance (HIPAA, SOC 2)	Audit Manager + Config
Third-party datasets, revision notifications	AWS Data Exchange
PII in S3	Amazon Macie
Who called which API	AWS CloudTrail
Configuration compliance	AWS Config
Vulnerability scanning	Amazon Inspector
Best-practice recommendations	AWS Trusted Advisor
Private connection to Bedrock	AWS PrivateLink
S3 or DynamoDB privately from a VPC, free	Gateway endpoint
Encryption keys	AWS KMS
Store and auto-rotate API keys or credentials	Secrets Manager

Tricky practice questions and traps

Each row is a question type that commonly trips people up on practice tests, with the rule that answers it.

Question type	Rule
AI vs. generative AI	New content → generative AI; label or number → traditional ML
Precision, F2: before or after training?	After training, on test data; they come from the confusion matrix

Question type	Rule
Lifecycle ordering	Business goal → frame problem (define error metric) → data → model
Review privacy laws and ethics before any technical work	Business goal identification
Stockouts or overstock from bad demand predictions	Data collection
Risk and reward	Reinforcement learning; human rankings → RLHF
Match prompting techniques	Several examples → few-shot; break into steps → chain-of-thought; no examples → zero-shot
Break a problem into smaller tasks solved in sequence	Least-to-most prompting
Step-by-step explanations	Chain-of-thought
Keep chatbot tone and style, no human, cost-effective	Prompt engineering with positive prompts; RLHF needs humans and costs more
Same prompt must give consistent output	Lower temperature
Reduce hallucinations	Lower temperature, plus RAG for grounding
Stable Diffusion images too generic	Raise the classifier-free guidance (CFG) scale
Red team bypassing safety to expose PII	Jailbreak
Gen AI-specific challenges	Toxicity, hallucination, intellectual property
Drawbacks of generative AI (Select TWO)	Hallucination, knowledge cutoff
Gen AI capabilities	Personalization, adaptability, scalability; not metrics
LLM vs. foundation model	Text → LLM; fine-tune for many tasks → foundation model
FM capability matching	Pick the broad capability, not a narrow application

Question type	Rule
Assessing text, image or both outputs	Modality
Traditional ML vs. generative AI	Translating documents with embedded images → generative AI; churn prediction and sentiment → traditional ML; designs from text → generative AI
Adapt pre-trained model to industry terms with labeled data	Fine-tuning
Primary benefit of fine-tuning	Better performance for specific use cases (faster training is secondary)
Fine-tuned FM below target accuracy	Increase epochCount
Regularly update an FM with new unlabeled data	Continued pre-training
Bedrock customization steps in order	Prepare dataset → create fine-tuning or pre-training job → purchase Provisioned Throughput
Occasional, unpredictable use, pay-as-you-go	On-Demand
Fine-tuned custom model, consistent traffic	Provisioned Throughput
Cut cost of a 20-example few-shot prompt	Reduce tokens (fewer examples, shorter output)
What drives inference latency (Select TWO)	Length of input and length of generated output
Responsiveness or runtime efficiency	Average response time
Evolving questions, low cost	RAG plus prompt engineering, not retraining
Chatbot over Kendra-indexed policy docs, most cost-efficient	RAG, not retraining or Lex
Store and query embeddings for fast vector search	OpenSearch Service (preferred over Aurora pgvector; never Redshift or plain RDS)
Vector search services (Select THREE)	OpenSearch, DocumentDB, Neptune (also pgvector, MemoryDB)

Question type	Rule
Interpretable, hierarchical breakdown of factors	Decision trees
Predict seashell type from size, shape, texture	k-NN (supervised); k-means is clustering
Segment patients into groups, no labels	k-means (RCF = anomalies, PCA = fewer features, KNN = labeled classification)
Unsupervised approaches	Group users → clustering; shows watched together → association rules; how preferences spread → probability density
Upload an image to a deployed endpoint for predictions	Inference
Visualize multiclass performance, defective vs. non-defective	Confusion matrix
Proportion of correct predictions, balanced classes	Accuracy
Rare fraud, binary probabilities, distinguish classes	AUC-ROC
Benchmark a fine-tuned LLM's accuracy	F1 score
Biased, skewed, unbalanced training data	Data augmentation
Skewed dataset missing demographic groups	Fairness
Over-represented groups in training data	Sampling bias
Clarity on how eligibility decisions are made	Transparency
Detect unintended bias in an LLM	SageMaker Clarify
Explain feature effects to stakeholders	Partial dependence plots (Clarify)
Control chatbot output for accuracy and compliance	Guardrails for Amazon Bedrock
Content types Guardrails filters (Select TWO)	Hate, violence

Question type	Rule
Guardrails filter matching	Content filters = violence, insults, prompt attacks; denied topics = forbidden subjects; word filters = specific terms; contextual grounding = unsupported claims
Detect toxic text in comments, no labeled data	Comprehend toxicity detection
Filter explicit images and video, low overhead	Rekognition Content Moderation
Chatbot across text and voice with conversation insights	Amazon Lex
Sentiment with rule-based NLP and gen AI (Select TWO)	Comprehend + Bedrock
Summarize structured patient data	Comprehend Medical + rule-based logic
Speech → transcript + summary of patient visits	AWS HealthScribe
Turn messy records into structured data for ML	AWS Glue
Pre-built solutions and models to deploy	SageMaker JumpStart
No-code training, image labeling, pre-built vision models	Canvas, Ground Truth, JumpStart
Log and audit API calls, which identities called Bedrock	CloudTrail
Save detailed input and output of Bedrock calls	Bedrock model invocation logging
Log Bedrock and Lambda calls, keep 3 years cheaply (Select TWO)	CloudTrail + S3 Intelligent-Tiering (Tutorials Dojo's answer)
Regulators want official AWS compliance reports	AWS Artifact
Third-party compliance datasets, notify on new revisions	AWS Data Exchange

Question type	Rule
Continuously monitor HIPAA compliance	Audit Manager + Config
Store and auto-rotate API keys for Lambda	Secrets Manager
Secure training data (Select TWO)	Role-based access + cryptographic hashing
Security capability matching	Data protection = control of data; threat detection = real-time monitoring; application security = vulnerabilities across development; infrastructure protection = networks, endpoints
SageMaker notebooks reach S3 without leaving AWS, cheapest	VPC gateway endpoint for S3
Fine-tune in-Region with no internet exposure (Select TWO)	Bedrock fine-tuning + VPC with PrivateLink (not NAT Gateway)

Traps that recur

- **Bait words:** a wrong option repeats the question's keyword ("bias", "audit", "log") but names the wrong service.
- **Noise:** extra details (BERT, poisoning, Feature Store) pull you away from the real requirement.
- **Capability vs. metric:** a positive-sounding measurement is not a capability.
- **Labeled vs. unlabeled:** labeled examples → fine-tuning; unlabeled domain data → continued pre-training.
- **Decided vs. measured:** success metrics are chosen in problem framing, measured in evaluation.
- **Artifact vs. Audit Manager:** AWS's reports vs. evidence about your own account.
- **Ground Truth vs. A2I:** humans before training (labels) vs. after deployment (reviews).
- **Clarify vs. Model Monitor:** bias and explanations vs. production drift.
- **Accuracy on rare classes:** misleading; use recall, precision, F1 or AUC. With balanced classes and overall correctness, accuracy is right.
- **k-NN vs. k-means:** predict a named type → k-NN; group without labels → k-means.
- **Textract vs. Rekognition:** documents and handwriting → Textract; faces, objects, scenes → Rekognition.

- **Secrets Manager vs. KMS:** store and rotate secrets → Secrets Manager; encryption keys → KMS.
- **PartyRock vs. JumpStart:** playground with no AWS account vs. catalog of deployable pre-trained models.
- **Medical input type:** speech → HealthScribe; text → Comprehend Medical; scans → Textract.
- **Matching questions:** read each description first, then pick the answer.
- **Pace:** budget about 80 seconds a question; rushing turns services you know into misses.

Exam-day strategy

Read slowly, find the keyword, and flag rather than stall; 90 minutes for 65 questions is about 80 seconds each.

Answering

1. Read the last sentence first: what is actually asked, and is it Select TWO or THREE?
2. Name the task in five words and predict the service before reading the options.
3. Underline the keyword (handwriting, fairness, no examples, unlabeled, AWS's own reports).
4. Eliminate answers from the wrong family (a metric when asked for a capability, S3 for vector search).
5. Between two close answers, pick the most managed, least effort option unless the question asks for control.
6. Flag anything over 90 seconds and move on; answer every question, since guessing costs nothing.

Online (OnVUE) exam check

- Run the Pearson VUE system test on the exam computer a day ahead.
- Check in 30 minutes early with a government photo ID.
- Clear desk, closed door, no second monitor, phone out of reach.
- Don't photograph the screen, including the results screen; it breaks testing rules.

The night before

- Reread section 9 and the service picker, nothing new.
- Sleep; a rested read of the keyword is worth more than one more practice set.